

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

1. IDENTIFICACIÓN DE LA GUÍA

Nombre de la guía:	Desarrollo de servicios web con persistencia en bases de datos utilizando FastAPI y SQLAlchemy
Código de la guía (No.):	002
Taller(es) o Laboratorio(s) aplicable(s):	Laboratorio Devops
Tiempo de trabajo práctico estimado:	4 días
Asignatura(s) aplicable(s):	Aplicaciones y Servicios Web
Programa(s) Académico(s) / Facultad(es):	Tecnología en Desarrollo de software

COMPETENCIAS	CONTENIDO TEMÁTICO	INDICADOR DE LOGRO
Diseñar y desarrollar servicios web utilizando herramientas modernas de programación, aplicando estructuras de datos y validación de información para el intercambio de datos en formato JSON.	Arquitectura de servicios web por capas (models, schemas, crud, api, db) Conexión a bases de datos relacionales (PostgreSQL) Uso de ORM con SQLAlchemy Modelado de datos en bases de datos Definición de schemas con Pydantic Implementación de operaciones CRUD Uso de inyección de dependencias en FastAPI Organización de endpoints mediante routers Persistencia de datos Control de versiones con Git y GitHub	El estudiante implementa un servicio web funcional con persistencia en base de datos, estructurado en capas, que permite realizar operaciones CRUD mediante FastAPI, utilizando SQLAlchemy para la gestión de datos, Pydantic para validación y GitHub para el trabajo colaborativo.

2. FUNDAMENTO TEÓRICO

En el desarrollo de aplicaciones modernas, los servicios web no solo permiten la comunicación entre sistemas, sino que también requieren mecanismos robustos para la **gestión y persistencia de la información**. En el Taller 1 se abordó la construcción de servicios web básicos utilizando FastAPI, con almacenamiento temporal en memoria. Sin embargo, este enfoque no es adecuado para aplicaciones reales, ya que la información se pierde al reiniciar el sistema.

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

En este contexto, surge la necesidad de integrar una **base de datos relacional**, como PostgreSQL, que permita almacenar la información de manera persistente, garantizando su disponibilidad y consistencia en el tiempo.

Para facilitar la interacción entre el código y la base de datos, se utiliza un **ORM (Object-Relational Mapping)**. Un ORM permite mapear tablas de la base de datos a clases en el lenguaje de programación, lo que simplifica las operaciones de consulta, inserción, actualización y eliminación de datos. En el ecosistema de Python, una de las herramientas más utilizadas para este propósito es **SQLAlchemy**, la cual permite definir modelos que representan las tablas de la base de datos y manipular los datos mediante objetos.

Adicionalmente, en aplicaciones basadas en FastAPI es fundamental gestionar adecuadamente las conexiones a la base de datos. Para ello se utiliza el concepto de **inyección de dependencias**, el cual permite proporcionar de manera controlada recursos como sesiones de base de datos a los diferentes endpoints de la API. Este enfoque mejora la organización del código, facilita las pruebas y evita problemas asociados al manejo incorrecto de conexiones.

Otro aspecto importante en el desarrollo de servicios web escalables es la **organización modular del código**. En lugar de concentrar toda la lógica en un solo archivo, se emplea una arquitectura por capas que separa responsabilidades en distintos componentes, tales como:

- **models**: representan las tablas de la base de datos mediante SQLAlchemy
- **schemas**: definen la estructura de los datos de entrada y salida utilizando Pydantic
- **crud**: contienen la lógica para interactuar con la base de datos
- **api**: definen los endpoints del servicio web mediante routers
- **db**: gestiona la conexión a la base de datos

El uso de **routers** en FastAPI permite organizar los endpoints en módulos independientes, facilitando la escalabilidad y el mantenimiento del sistema.

En este tipo de aplicaciones, el intercambio de información continúa realizándose mediante **JSON**, pero ahora los datos ya no se almacenan en memoria, sino que se persisten en una base de datos, lo que permite construir aplicaciones más cercanas a entornos reales.

Finalmente, el desarrollo colaborativo sigue siendo un componente clave del proceso. El uso de herramientas como **Git y GitHub** permite gestionar el código fuente, controlar versiones y coordinar el trabajo en equipo, especialmente en proyectos donde múltiples componentes deben integrarse en una misma arquitectura.

En esta guía práctica, los estudiantes desarrollarán un servicio web con FastAPI que implementa un **CRUD completo con persistencia en PostgreSQL**, utilizando SQLAlchemy como ORM, aplicando inyección de dependencias para la gestión de la base de datos y organizando el código en una arquitectura modular basada en routers y separación por capas.

3. OBJETIVOS

Objetivo general

Desarrollar un servicio web utilizando FastAPI que implemente un CRUD completo con persistencia en una base de datos PostgreSQL, aplicando una arquitectura organizada por capas mediante el uso de SQLAlchemy, Pydantic, inyección de dependencias y routers.

Objetivos específicos

1. Implementar la conexión a una base de datos PostgreSQL utilizando SQLAlchemy.

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

2. Definir modelos de base de datos que representen las entidades del sistema de laboratorios, utilizando SQLAlchemy.
3. Diseñar esquemas de datos utilizando Pydantic para validar la información de entrada y salida del servicio web.
4. Implementar operaciones CRUD (crear, consultar, actualizar y eliminar) para las entidades del sistema.
5. Aplicar el patrón de separación por capas mediante la organización del código en módulos: models, schemas, crud, api y db.
6. Utilizar inyección de dependencias para gestionar correctamente las sesiones de base de datos en los endpoints.
7. Implementar endpoints REST organizados mediante routers en FastAPI.
8. Verificar el funcionamiento del servicio web mediante pruebas en la interfaz interactiva de FastAPI (Swagger).
9. Gestionar el desarrollo del proyecto de forma colaborativa utilizando Git y GitHub.

4. RECURSOS REQUERIDOS

Para el desarrollo de esta práctica se requieren los siguientes recursos:

Equipos

- Computador personal o estación de trabajo por estudiante.

Herramientas de software

- Sistema operativo Linux o Windows.
- Python 3.10 o superior.
- FastAPI.
- Git.
- PostgreSQL
- SQLAlchemy
- Uvicorn
- Cuenta en GitHub.
- Editor de código (por ejemplo: Visual Studio Code).

Material bibliográfico y recursos digitales

- Documentación oficial de FastAPI.
- Documentación oficial de Pydantic.
- Documentación oficial de Python.
- Guías básicas de uso de Git y GitHub.
- Repositorio de clase https://github.com/Juanmorales177809/apps_services.git

5. ASPECTOS DE SEGURIDAD

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

La práctica descrita en esta guía corresponde a una actividad de desarrollo de software, por lo cual no se identifican riesgos físicos o químicos asociados al uso de laboratorios experimentales.

Sin embargo, se recomienda tener en cuenta las siguientes consideraciones:

- Mantener una postura adecuada durante el uso prolongado del computador para evitar fatiga o lesiones musculares.
- Evitar la manipulación inadecuada de cables o conexiones eléctricas de los equipos.
- Realizar copias de seguridad periódicas del código desarrollado para evitar pérdida de información.

6. PROCEDIMIENTO O METODOLOGÍA PARA EL DESARROLLO

La práctica se desarrollará en equipos de trabajo conformados por dos a tres estudiantes.

Actividad 1: Configuración inicial del proyecto

- Crear repositorio en GitHub
- Clonar repositorio
- Crear entorno virtual
- Activarlo
- Instalar dependencias:
 - Fastapi
 - Uvicorn
 - SQLAlchemy
 - psycopg2-binary
- Crear:
 - .gitignore
 - requirements.txt

Actividad 2: Comprensión del problema y del modelo de datos

A. Planteamiento del problema

En el contexto de los laboratorios de la institución, es necesario contar con un sistema que permita gestionar la información del personal que participa en las actividades académicas y técnicas, así como su formación académica y profesional.

Actualmente, la información del personal y su formación se encuentra distribuida en diferentes medios, lo que dificulta su consulta, actualización y control. Por esta razón, se requiere desarrollar un sistema que permita centralizar esta información y facilitar su gestión.

Se requiere desarrollar un servicio web que permita gestionar la información del personal y su formación, utilizando una **base de datos relacional PostgreSQL** para garantizar la persistencia de los datos.

El sistema debe permitir:

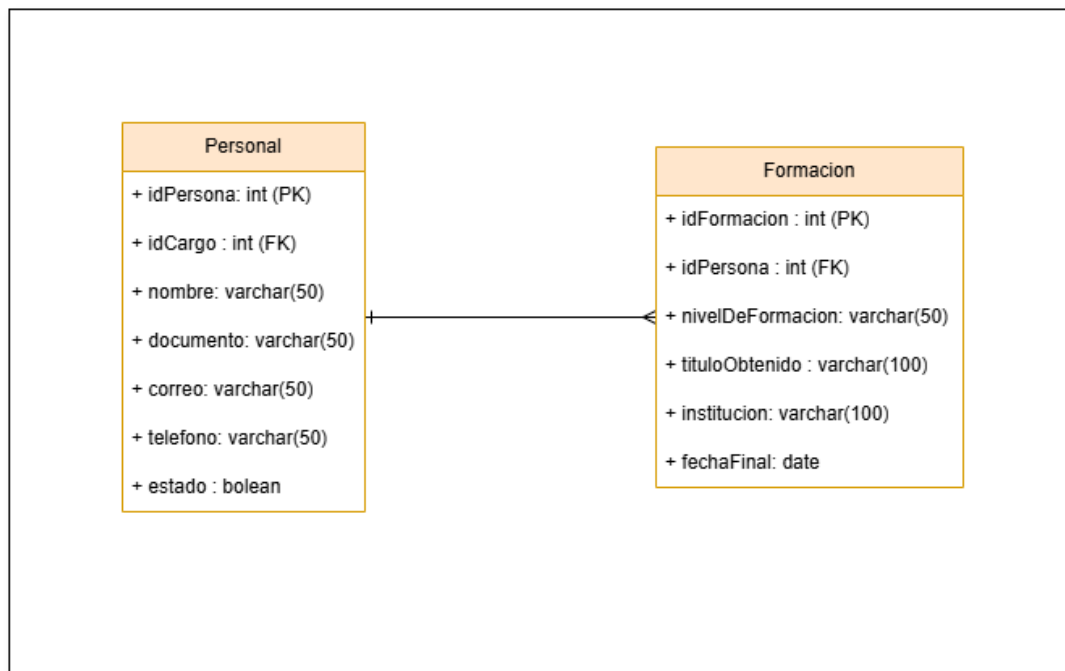
- Registrar información del personal del laboratorio.
- Consultar los registros existentes.
- Actualizar la información del personal.
- Eliminar registros cuando sea necesario.
- Registrar la formación académica asociada a cada persona.
- Consultar, actualizar y eliminar la información de formación.

Para lograr esto, es necesario diseñar e implementar un servicio web que siga una **arquitectura organizada por capas**, que se conecte a una base de datos real y que permita realizar operaciones CRUD completas sobre las entidades definidas.

Cada equipo de trabajo desarrollará este servicio utilizando FastAPI, SQLAlchemy y PostgreSQL, trabajando sobre un esquema de base de datos asignado por el docente, garantizando así el aislamiento entre los diferentes grupos.

Modelo de datos del sistema

A continuación, se presenta el diseño de la base de datos que será utilizado en este taller:



El sistema está compuesto por dos tablas principales:

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

Tabla Personal

Almacena la información general de las personas vinculadas a los laboratorios.

Campos principales:

- idPersona: identificador único de la persona (clave primaria)
- idCargo: identificador del cargo (referencia)
- nombre: nombre completo
- documento: número de identificación
- correo: correo electrónico
- telefono: número de contacto
- estado: indica si la persona se encuentra activa

Tabla Formacion

Almacena la información académica o profesional asociada a cada persona.

Campos principales:

- idFormacion: identificador único (clave primaria)
- idPersona: identifica a la persona asociada
- nivelDeFormacion: nivel académico (técnico, profesional, etc.)
- tituloObtenido: título alcanzado
- institucion: institución donde se obtuvo el título
- fechaFinal: fecha de finalización

Relación entre las tablas (conceptual)

- Una persona puede tener múltiples registros de formación.
- Cada registro de formación pertenece a una única persona.

Nota: En este taller, esta relación se manejará únicamente mediante el campo idPersona. No se requiere implementar relaciones complejas en el código (por ejemplo, joins o respuestas anidadas).

El objetivo de esta actividad es comprender el modelo de datos antes de su implementación en código.

Actividad 3: Configuración de la base de datos y modelos

El equipo deberá configurar la conexión a la base de datos PostgreSQL y definir los modelos correspondientes a las tablas del sistema.

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

Nota: Las tablas deben crearse dentro del schema asignado por el docente.

Actividades a realizar:

1. Crear el archivo db.py para la conexión a la base de datos.
2. Configurar:
 - motor de conexión (engine)
 - sesión de base de datos
 - función get_db() para inyección de dependencias
3. Conectarse al schema asignado por el docente.
4. Crear los modelos SQLAlchemy:
 - Personal
 - Formacion
5. Definir correctamente:
 - nombre de la tabla
 - tipos de datos
 - llaves primarias
 - llaves foráneas (idPersona en Formacion)

Actividad 4: Construcción del backend (CRUD y API)

El equipo deberá desarrollar el backend completo del sistema implementando la arquitectura por capas.

Actividades a realizar:

4.1 Schemas

1. Crear los esquemas Pydantic para cada entidad:

Para Personal:

- PersonalCreate
- PersonalUpdate
- PersonalOut

Para Formacion:

- FormacionCreate
- FormacionUpdate
- FormacionOut

4.2 CRUD

2. Implementar funciones CRUD para cada entidad:

Para Personal:

 Institución Universitaria Reacreditada en Alta Calidad	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

- crear
- listar
- consultar por id
- actualizar
- eliminar

Para Formacion:

- crear
- listar
- consultar por id
- actualizar
- eliminar

4.3 API (routers)

3. Crear los endpoints organizados mediante routers:

Para Personal:

- POST /personal
- GET /personal
- GET /personal/{id}
- PUT /personal/{id}
- DELETE /personal/{id}

Para Formacion:

- POST /formacion
 - GET /formacion
 - GET /formacion/{id}
 - PUT /formacion/{id}
 - DELETE /formacion/{id}
4. Utilizar inyección de dependencias (Depends(get_db)) en los endpoints.

Actividad 5: Pruebas del sistema

El equipo deberá verificar el funcionamiento del sistema mediante pruebas en la interfaz de FastAPI.

Actividades a realizar:

1. Ejecutar el servidor y acceder a la interfaz Swagger (/docs).
2. Realizar pruebas completas del CRUD:

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

- Crear registros en Personal y Formacion
- Consultar registros
- Consultar por id
- Actualizar registros
- Eliminar registros

Actividad 6: Trabajo colaborativo

El equipo deberá evidenciar el trabajo colaborativo mediante el uso de Git y GitHub.

Actividades a realizar:

1. Cada integrante debe realizar al menos un commit desde su equipo local.
2. Los commits deben incluir mensajes claros que describan el aporte realizado.
3. El repositorio debe reflejar el trabajo distribuido del equipo.

Ejemplos de mensajes:

- Aporte: configuración de base de datos y db.py
- Aporte: modelos SQLAlchemy
- Aporte: schemas Pydantic
- Aporte: funciones CRUD
- Aporte: endpoints API

Resultado esperado del taller

Al finalizar la práctica, el equipo deberá contar con un servicio web funcional que:

- se conecta a una base de datos PostgreSQL
- implementa un CRUD completo
- sigue una arquitectura organizada por capas
- permite gestionar la información de personal y formación
- está versionado y documentado en GitHub

7. PARÁMETROS PARA ELABORACIÓN DEL INFORME

El informe del taller deberá presentarse en formato **README.md** dentro del repositorio del proyecto en GitHub.

El repositorio debe contener tanto el código fuente del sistema como la documentación del proyecto, garantizando su correcta ejecución y comprensión.

Contenido del README.md

El archivo README.md debe incluir como mínimo los siguientes apartados:

 Institución Universitaria Reacreditada en Alta Calidad	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

1. Información general

- Nombre del proyecto
- Integrantes del equipo
- Asignatura
- Fecha

2. Descripción del sistema

- Descripción general del sistema desarrollado
- Entidades implementadas (Personal y Formación)
- Descripción de la arquitectura utilizada (models, schemas, crud, api, db)

3. Configuración del entorno

- Creación del entorno virtual
- Activación del entorno
- Instalación de dependencias
- Uso del archivo requirements.txt

4. Configuración de la base de datos

- Descripción de la conexión a PostgreSQL
- Uso del archivo db.py
- Schema asignado (sin incluir credenciales sensibles)

5. Ejecución del proyecto

- Comando para ejecutar el servidor (ej. `uvicorn main:app --reload`)
- URL de acceso a la API
- Acceso a la documentación Swagger (`/docs`)

6. Endpoints implementados

Listado de endpoints del sistema, por ejemplo:

Personal

- POST `/personal`
- GET `/personal`
- GET `/personal/{id}`
- PUT `/personal/{id}`
- DELETE `/personal/{id}`

Formacion

- POST `/formacion`
- GET `/formacion`
- GET `/formacion/{id}`

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

- PUT /formacion/{id}
- DELETE /formacion/{id}

7. Evidencias de funcionamiento

El README debe incluir capturas de pantalla o evidencias de:

- Ejecución del servidor
- Uso de Swagger
- Pruebas de los endpoints (POST, GET, PUT, DELETE)
- Evidencia de persistencia en la base de datos

8. Control de versiones

- Enlace al repositorio en GitHub
- Evidencia de commits realizados por cada integrante
- Descripción breve del aporte de cada miembro

9. Conclusiones

- Principales aprendizajes
- Dificultades encontradas
- Soluciones aplicadas

Nota: El README debe permitir que cualquier persona pueda ejecutar el proyecto sin necesidad de información adicional.

Requisitos del repositorio

El repositorio debe:

- Contener el código fuente completo del proyecto
- Incluir el archivo README.md correctamente estructurado
- Incluir el archivo requirements.txt
- Incluir el archivo .gitignore
- Estar organizado de acuerdo con la arquitectura propuesta

Nota importante

No se aceptarán informes en formatos externos (PDF, Word, etc.). Toda la documentación debe estar incluida dentro del repositorio en el archivo README.md.

8. DISPOSICIÓN DE RESIDUOS

La actividad descrita en esta guía no genera residuos físicos o químicos. En consecuencia, no se requiere un procedimiento específico de disposición de residuos para esta práctica.

9. BIBLIOGRAFÍA

FastAPI. (2024). *FastAPI Documentation*. <https://fastapi.tiangolo.com>

Pydantic. (2024). *Pydantic Documentation*. <https://docs.pydantic.dev>

Python Software Foundation. (2024). *Python Documentation*. <https://docs.python.org>

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

Chacon, S., & Straub, B. (2014). *Pro Git*. Apress.

Fielding, R. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine.

Elaborado por:	<i>Juan Carlos Morales Guerra</i>
Revisado por:	<i>Juan Carlos Morales Guerra</i>
Versión:	001
Fecha:	27-02-2026